Page · Tag · 1 backlink

You: Hi! I am working on this personal project. There's just me, no clients, users or collaborators. I learning about data analysis and i started this project but now i realise there's is so much i don't know about db and data analisys. So, before i continue with my project, i need to reverse engineer my curent status and correct the approach. I presume that this particular project is really nothing new, so there must exist documentation, strategies and playbooks for it. I need you to help me with:

1. precisely define my project according to professional data analysis frameworks.
2. A list with standard issues and problems that exist with this type of project. Like: The "Repeated Alert" Phenomenon, The Multi-Job Problem, Keys, The "Source of Truth" Problem, Join Logic and so on!
3. 3. I need to re-analyse my project from scratch and verify each step such that i will not have to restart again and affect issues that were solved any time a new issue appears. I need to understand the terminology and practices for my particular type of project.

Leo AI: Yes, this makes complete sense. What you're describing is called **reverse-engineering your data architecture** — auditing a project that grew faster than your understanding of it, then rebuilding the mental model before touching code again.

Let me start with part 1, then we'll pause so you can take notes.

---

1. Defining your project in professional terms

Your project has a formal name: a **BI (Business Intelligence) analytics solution** built on a **star-schema semantic layer**.

Let's break down what each piece of your stack actually *is*, using industry vocabulary:

Your component	Professional term
Postgres (`london_job_market`)	The **source system / system of record** — where raw truth lives
`companies_sic_flat`	A **denormalized fact table** (or "wide table") — one row per company, pre-joined for speed
`company_sic`	A **bridge/junction table** — resolves many-to-many relationships (one company ↔ many SIC codes)
`DateTable`	A **date spine / calendar dimension** — the backbone of time intelligence
`job_alerts`	An **event/fact table** — rows are events (alerts), not entities
`companies`	A **dimension table** — descriptive attributes of an entity
Power BI model	The **semantic layer** — business logic expressed as measures and relationships
Threshold table + filter measure	A **parameter-driven visual filter** (a standard UX pattern)

The framework name you should search for is **Kimball dimensional modeling** (the classic methodology for exactly this kind of project). The other half of your discipline is **data quality management** — which is where your blockers live.

One important classification: your project mixes **two grains**:

- **Company grain** — "how many companies exist per grid cell" (static-ish)
- **Alert grain** — "when did alerts arrive" (time-series events)

Most of your v4.0 pain (inactive relationships, fan-out confusion, date spine conflicts) comes from managing two grains in one model. That's normal, but it must be done deliberately.

Before I give you part 2 (the catalog of standard problems like Repeated Alerts, Multi-Job, Source of Truth): does the grain concept above make sense? Tell me when you've got it noted, and ask anything about part 1 — then we move to the problem catalog.

You: Let me try to define, in my own plain words what i am looking for with this project. I have to sources of data. 1. The companies house database and 2. my own

email alerts from indeed and Linked in (possible other small sources, not important now.) My goals are:

1. visualise distribution of companies on a map by domain (ex: Advertising, Solicitors, etc), by Incorporation date and by geolocation / district. - This goal was reached and verified.
2. Visualise job alerts distributions on a map by matching companies in the db with companies names in the job alerts. Create charts with the job alerts by time or domain (job title) or companies to see the flow and find patterns (job-alerts that repeat in intervals > 30 days). - The scope is to identify fake jobs and also find viable companies that hire based on how long these companies existed and if jobs alerts are matched. My thinking was that for instance, because i have Recruitment agencies in my db (actually 'human resources' - there is no recruitment category in Companies House db.) - i should be able than to see a bubble with all the job alerts that match with recruitment companies, or just companies sending job-alerst by themselfs. does this make sense?

Leo AI: It makes sense, but there are two critical misunderstandings in your logic that will cause you pain if not corrected now. Let me address them plainly.

---

1. The "Recruitment Agency" Problem

You said: *"I should be able to see job alerts that match with recruitment companies."*

This is backwards. In the UK job market:

- **Recruitment agencies** (Hays, Reed, etc.) are **not employers**. They post jobs on behalf of clients.
- If you match alerts to recruitment companies, you will see **massive bubbles of noise** — hundreds of alerts from companies that don't actually hire anyone. They're middlemen.
- What you *want* is the **reverse**: when an alert comes in, identify which recruitment agency posted it, then **attribute the alert to their client company** (if that data is available in the alert text).

Most Indeed/LinkedIn alerts **do not contain the end-client name** — only the agency name. So your "fake job detection" logic cannot rely on matching to recruitment companies. It must rely on **frequency analysis** (alerts from the same agency for the same role, repeated at regular intervals).

---

2. The "Fake Job" Detection Fallacy

You said: *"Scope is to identify fake jobs... and find viable companies that hire."*

Here's the hard truth: **You cannot identify fake jobs from this data alone.**

What you *can* identify:

- **Bot-like behavior**: Same company posting >3 alerts for the same role within 7 days
- **Agency fatigue**: A recruitment agency posting 50+ alerts/month (normal for them, suspicious if flagged as "employer")
- **Ghost companies**: Companies with 0 alerts but 100+ job applications (if you ever add outbound data)

What you *cannot* identify:

- Whether a job posting is a real vacancy or a "zombie posting" (stale, never filled)
- Whether a company is fraudulent (requires Companies House financials, not job alerts)
- Whether an alert is "fake" (a scam) — that requires content analysis of the job description

Be honest about what your data can prove. You can detect *patterns*, not *truth*.

---

3. The Grain Problem (Again)

Your two goals operate on **different grains**:

Goal	Grain	Primary Key	Time Dimension
Company distribution	Company	`company_number`	`incorporation_date`
Alert pattern analysis	Alert	`alert_id`	`date_received`

These two grains **cannot share a single fact table**. You already know this from v4.0, but let me restate it in plain English:

- A **company** exists once. It has one `incorporation_date`. It may have 4 SIC codes.
- An **alert** exists once. It has one `date_received`. It may match to 0 or 1 company. It may have 0 or 1 job title.

If you try to flatten these into one table, you get **fan-out duplication** — one company × 100 alerts = 100 rows for the same company. That's your `companies_sic_flat` problem. That's your memory crash.

The professional solution is **two separate fact tables** connected to the same dimension tables (companies, dates, SIC codes). This is called a **snowflake** or **fact constellation** schema.

---

4. The Matching Problem (The Hardest Part)

You said: *"matching companies in the db with companies names in the job alerts."*

This is **not easy**. Here's why:

- Companies House: "REED EDUCATION LIMITED"
- Indeed alert: "Reed Education"
- LinkedIn alert: "Reed Student Teachers"
- Another company: "REED & CO SOLICITORS LLP"

Exact string matching will fail. You need a **fuzzy matching strategy** (Levenshtein distance, Jaro-Winkler, or token-based similarity). Without it, your match rate will be 30–50%, and you'll waste time chasing false negatives.

The professional term for this is **entity resolution** or **record linkage**. It's a known unsolved problem in data engineering. You should expect 40–60% match rate and plan your analysis around the unmatched 40–60%.

Does this land? Tell me which of these four points surprised you most, and we'll move to part 2 (the problem catalog) — but now I'll frame it in the context of what you just said.

You: this is very interesting and informing. thank you. Actually i do get a 'Running fuzzy company match ...Match rate: 47.9%'. Again, this is a personal project, so it's more like exploring and learning than getting back precise, professional results. But also, i want to learn how to do this properly, so this is what i am looking for: A proper way to prepare and understand the data before i start so i can use PowerBI to visualise the data instead of trying to fix it. One important issue is that my data is not really clean. The house company db is quite large and fuzzy, although i am only using London data, so it's manageable and it mostly working and verified. But the job-alerts data is kind of bad. Never the less, i believe my BI (Business Intelligence) analytics project built on a star-schema semantic layer is not the first one, so I'm sure this has been studies and there must be standard solutions. There's must be companies outhere doing this kind of analises all the time....

Leo AI: You're right — companies do this all the time. The professional term for your project is **a multi-source entity resolution and time-series analytics pipeline**. Here's the catalog of standard problems you'll face, and how the industry solves them.

---

2. Standard Problems & Professional Solutions

The "Repeated Alert" Phenomenon

Problem: Recruitment agencies post the same role repeatedly. You see "3 alerts for 'Junior Developer' from Reed in 30 days" and think it's a pattern. It's not — it's noise.

Solution: Define a **deduplication window**. If the same `company_name` (matched) posts the same `job_title` (normalized) within N days, collapse them into one "campaign." The industry standard is **30-day rolling window** for agency roles, **90 days** for executive roles.

Term: *Alert deduplication / campaign grouping*

The Multi-Job Problem

Problem: A single job posting appears on Indeed, LinkedIn, and the company website. You count 3 "jobs" when there's really 1.

Solution: You cannot reliably deduplicate across sources without a **unique job ID**. Since you don't have one, use **fuzzy matching on (company + job_title + location + posted_date)** within a 3-day window. If all four match, treat as duplicate.

Term: *Cross-source entity deduplication*

The Source of Truth Problem

Problem: Companies House says company X was incorporated 2015. Indeed says company X posted a job in 2026. LinkedIn says company X was founded 2020. Which is right?

Solution: Every source has its own truth. Companies House is truth for legal existence. Job platforms are truth for hiring activity. They answer different questions. Never try to reconcile them into one "founded date."

Term: *Source-specific provenance / single-attribute truth*

Join Logic & Fan-Out

Problem: One company has 4 SIC codes. You join it to a fact table and get 4x rows. Your map shows 4x bubbles. Memory crashes.

Solution: Never join dimension attributes into a fact table grain. Keep `companies_sic_flat` as a *denormalized dimension* (one row per company, with all SIC codes in one row). Keep `company_sic` as a *separate bridge table* for filtering. The fact table (alerts) joins only to `companies[company_number]`, not to SIC codes.

Term: *Dimensional modeling / bridge tables / fact-dimension separation*

The Key Problem

Problem: `company_number` is the perfect key for Companies House. But job alerts have `company_name`, not `company_number`. You're joining on fuzzy text.

Solution: Accept that **your join key is probabilistic, not deterministic**. Build a **match confidence score**:

- Exact match → confidence 1.0
- Fuzzy match (score > 0.85) → confidence 0.8
- No match → confidence 0.0

Then **filter or annotate** by confidence. Don't treat 0.8 matches the same as 1.0 matches.

Term: *Probabilistic record linkage / match confidence scoring*

The Date Spine Conflict

Problem: You have `incorporation_date` (company dimension) and `date_received` (alert fact). Two dates, one model. Power BI gets confused about which relationship to follow.

Solution: One active date relationship (usually the fact table's date). The dimension's date (`incorporation_date`) uses an **inactive relationship** and is accessed only via `USERELATIONSHIP()` in DAX measures. This is standard practice.

Term: *Single active date spine / inactive relationship pattern*

The "Unmatched" Problem

Problem: 52% of your job alerts don't match to any company. You ignore them. You lose insight.

Solution: Analyze unmatched alerts separately. They tell you about:

- Companies not in Companies House (foreign, unregistered, scam)
- Fuzzy match gaps (your matching logic needs tuning)
- New companies too new to be registered

Term: *Unmatched entity analysis / null-attribution*

The Visual Density Problem

Problem: 78,000 grid cells on one map. Power BI crashes.

Solution: Progressive disclosure. Default view: borough-level aggregates (~33 tiles). Drill-down: ward-level (~2,000 tiles). Click: grid-cell detail (~5,000 tiles). Never

render the full grain without a filter.

Term: *Progressive disclosure / aggregation hierarchy*

The Timestamp Trap

Problem: You use `ingested_at` (pipeline batch time) instead of `date_received` (actual alert time). Your charts show pipeline artifacts, not real patterns.

Solution: Never use pipeline timestamps for analysis. `ingested_at` is only for data quality monitoring (e.g., "was this batch loaded on time?"). `date_received` is the ONLY valid time dimension for alert analysis.

Term: *Event-time vs. processing-time semantics*

---

3. The Professional Checklist (Before You Start)

Here's what senior data engineers do before writing a single query:

1. **Define the grain** — What is one row? (Company? Alert?)
2. **Identify the keys** — Deterministic (company_number) vs. probabilistic (company_name)
3. **Map the sources of truth** — Which source answers which question?
4. **Design the schema** — Star schema with separate fact tables, shared dimensions
5. **Plan the joins** — Which relationships are active? Which are inactive?
6. **Define deduplication rules** — When do two rows become one?
7. **Set match confidence thresholds** — What score is "good enough"?
8. **Plan aggregation levels** — What's the default view? What's the drill-down?
9. **Document the time semantics** — Event-time vs. processing-time
10. **Define success metrics** — What does "good enough" look like?

Does this catalog make sense? Which of these problems do you want to dive deeper into? Or should we move to part 3 — re-analyzing your project step-by-step to verify your current approach?

yes, thank you! i think we previously solved 'The "Repeated Alert" Phenomenon' - we have dedup for alerts that repeat <30 days, and 'The Multi-Job Problem Persists' and the key problem. I attached the parsing scripts and config files i hae (except 'soc_lookup.csv' - which is too big). I will add this 10 problems to a check list. Let's move to part 3. I am trying to understand the precise order of steps to approach the project and how can we make sure that once a step has been executed, there's no need to come back to it and fix it. Like verifying and

preparing the data to be certain there's no need to get back. This is one of my biggest issues. Every time i add something new, it affects every element in the project and i have to regress and fixed something i thought is done, or i wasn't even aware it exists. For instance, in v.3.0 each table had it's own 'data table' - now in v.4.0 we made a single data table, and i'm not sure if this actually improves or not the project.

This is the core pain point. In professional data engineering, this is called **Technical Debt** caused by **Procedural Coupling**. You are treating your data pipeline like a script that runs from top to bottom, where Step 4 depends on Step 3 being "perfect enough."

The professional solution is **Immutability and Idempotency**.

Here is the precise order of operations for your project, designed so that once a step is done, it never needs to change again.

---

3. The Verified Pipeline Order

Think of this as a factory assembly line. Once a product leaves Station 2, you never go back to Station 1. If Station 2 breaks, you fix Station 2, not Station 1.

Phase 1: Ingestion (The "Raw" Layer)

Goal: Capture data exactly as it arrives, timestamped with *when it arrived*, not *when it happened*. **Output:** 3 raw CSV/JSON files in a `raw/` folder.

1. `raw_companies.csv` (from Companies House)
2. `raw_alerts.csv` (from your email parser)
3. `raw_sic_codes.csv` (from your lookup)

Why this never changes: You don't clean here. You don't match. You don't deduplicate. You just save the text. If your parser breaks tomorrow, you re-run *only* Step 1. The rest of the pipeline is untouched.

Phase 2: Standardization (The "Clean" Layer)

Goal: Make every row look the same. Fix casing, remove HTML tags, normalize dates. **Output:** 3 standardized CSVs in a `clean/` folder.

1. `clean_companies.csv` (All names UPPER CASE, dates YYYY-MM-DD)
2. `clean_alerts.csv` (All names UPPER CASE, dates YYYY-MM-DD)
3. `clean_sic_codes.csv` (Standardized codes)

Why this never changes: This step has **no dependencies** on other data. It doesn't care about alerts or SIC codes. It only cares about one file at a time. Once `clean_companies.csv` exists, it is done. If you add a new SIC code later, you re-run *only* Step 2 for SIC codes. The companies file is untouched.

Phase 3: Entity Resolution (The "Match" Layer)

Goal: Link `clean_alerts` to `clean_companies`. **Output:** 1 table: `fact_alerts_matched.csv` **Columns:** `alert_id`, `date_received`, `job_title`, `company_name_in_alert`, `matched_company_number` (nullable), `match_confidence` (0.0–1.0)

Why this never changes: This step takes **only** `clean_alerts` and `clean_companies`. It does not touch SIC codes. It does not touch dates. It produces a lookup table. If your fuzzy match algorithm improves, you re-run *only* Step 3. The clean files are untouched. The fact that you now have a `match_confidence` score is your safety net — you can filter out low-confidence matches in Phase 4 without re-matching.

Phase 4: Dimensional Enrichment (The "Star Schema" Layer)

Goal: Attach descriptive attributes to the matched alerts. **Output:** 2 tables:

1. `dim_company_enriched.csv` (From `clean_companies` + `clean_sic_codes` joined on `company_number`)
2. `fact_alerts_enriched.csv` (From `fact_alerts_matched` joined to `dim_company_enriched` on `matched_company_number`)

Why this never changes: This step is **pure lookup**. It takes the output of Phase 3 and the output of Phase 2. It does not change the data; it only adds columns. If you add a new SIC code, you re-run the join in Step 4. The match data (Phase 3) is untouched.

Phase 5: Time Intelligence (The "Date" Layer)

Goal: Prepare the single `DateTable` for Power BI. **Output:** 1 table: `dim_date.csv`

Why this never changes: This is a static calendar. It never changes unless you add a new year. It is independent of all other data.

Phase 6: Aggregation & Deduplication (The "Analysis" Layer)

Goal: Apply your 30-day dedup window and create summary views. **Output:** 2 tables:

1. `fact_alerts_deduplicated.csv` (One row per "campaign")
2. `agg_company_hiring_activity.csv` (Monthly counts per company)

Why this never changes: This step uses **only** `fact_alerts_enriched`. It does not touch companies, SIC codes, or dates. If you change your dedup window from 30 to 45 days, you re-run *only* Step 6. The enrichment (Phase 4) is untouched.

Phase 7: Semantic Layer (Power BI Model)

Goal: Connect the tables. **Output:** Power BI `.pbix` file.

Why this never changes: This is just relationships. You connect `fact_alerts_enriched` to `dim_company_enriched`, `dim_date`, etc. If you add a new field in Phase 4, you just add it to Power BI. You never rewrite the relationships.

The Golden Rule: Idempotency

Idempotency means: If you run Step 3 twice, you get the **exact same result**. No duplicates, no errors.

To achieve this:

1. **Never modify a file in place.** Always write to a new file (e.g., `v1_clean.csv`, then `v2_clean.csv`).
2. **Use unique IDs.** Every row must have a primary key (`company_number`, `alert_id`).
3. Use `INSERT` not `UPDATE`. If you need to fix a match, add a new row with the correct match and mark the old one as `is_active = FALSE`. Never delete or overwrite.

Why Your v4.0 Single Date Table Was Correct

You asked: *"In v.3.0 each table had its own 'data table' - now in v.4.0 we made a single date table, and i'm not sure if this actually improves or not the project."*

The single `dim_date` table is the correct professional approach.

Here's why:

- **v3.0 (Multiple Date Tables):** You had `companies_date`, `alerts_date`, `sic_date`. This created **fan-out confusion** in Power BI. When you filtered by date, Power BI didn't know which date table to use. You got inactive relationship errors.
- **v4.0 (Single Date Table):** You have one `dim_date` that connects to `fact_alerts` (via `date_received`) and to `dim_company` (via `incorporation_date` using an inactive relationship). This is **standard Kimball dimensional modeling**. It works. It scales. It is correct.

Keep the single date table. It is not the problem. The problem was your **procedural coupling** (steps depending on each other in a fragile way), not your schema.

Your New Checklist (Before You Start Any Change)

Before you write a single line of code or change a relationship, ask:

1. **Does this change affect Phase 1?** (Raw data) → If yes, fix the parser.

2. **Does this change affect Phase 2?** (Standardization) → If yes, fix the cleaning logic.
3. **Does this change affect Phase 3?** (Matching) → If yes, fix the fuzzy match algorithm.
4. **Does this change affect Phase 4?** (Enrichment) → If yes, fix the join logic.
5. **Does this change affect Phase 5?** (Dates) → If yes, fix the calendar.
6. **Does this change affect Phase 6?** (Aggregation) → If yes, fix the dedup rule.

If the answer is "No" for all of the above, you are safe. You can proceed without fear of breaking previous steps.

Does this phased approach make sense? If so, your next step is to **document your current v4.0 tables** and map them to these 7 phases. Tell me which phase each of your current tables belongs to, and I'll help you identify where the coupling is happening.

You: yes. attached my TE3 export

Leo: This is the smoking gun. Your v4.0 model is not "almost right" — it is structurally inverted. You have built what professionals call a **spaghetti schema** (or anti-star): every relationship is Many-to-Many, and you've smuggled auto-date tables back in through the back door.

Let me map your actual model to the 7-phase pipeline so you can see exactly where the coupling lives.

Mapping Your Current Model to the Phases

Phase	What should exist	What you actually have	Verdict
1–2 Raw/Clean	Postgres tables	`companies`, `public job_alerts`	✅ Correct location
3 Match	One fact table with match scores	Inside `public job_alerts` (`match_confidence`, `matched_company_number`)	✅ Good — matching done upstream
4 Enrichment	Clean dimensions + one enriched fact	`companies_sic_flat`, `company_sic`, `dim_sic`, `ExcludedCcompanies`	⚠️ Two competing company tables
5 Date	ONE date table	`DateTable` **+ 3 hidden auto-date tables**	❌ Broken
6 Aggregation	Pre-aggregated summary tables	`companies_geo_clusters`	✅ Exists but misused
7 Semantic	Star schema, 1:*:* single-direction	8 relationships, **all Many-to-Many, all active**	❌ Root cause of everything

---

The Three Structural Defects

Defect 1: All-Eight Relationships Are Many-to-Many

Look at your `relationships.txt`. Every single one says `Many-to-Many; True`. In Kimball modeling, M2M is a **last resort**, used only for genuine bridge tables. You are using it as a default because Power BI *allows* it.

The damage: when every relationship is bidirectional-ish and ambiguous, filter direction becomes unpredictable. That's why `[Solicitors By Code]` vs `[Solicitors Correct]` gave you different numbers, why `Test A` ≠ `Test B`, and why `Bubble Count` counts rows from the wrong table entirely (`COUNTROWS('companies_sic_flat')` living inside `company_sic`?!).

The fix: One grain per table, one direction per relationship. `companies_sic_flat` should be **one row per company** (your denormalized dimension), and

`company_sic` should connect ONLY to `dim_sic` for filtering — never to the fact path.

Defect 2: The Auto-Date Tables Never Left

Your Best Practice Analyzer caught them: three `LocalDateTable_*` / `DateTableTemplate_*` calculated tables. These were auto-created in v3.0 when you had separate date columns, and they survived into v4.0. Worse, two are still wired up:

- `public job_alerts[ingested_at]` → auto-date table (**this is the Timestamp Trap** — you're analyzing pipeline time!)
- `companies[incorporation_date]` → another auto-date table

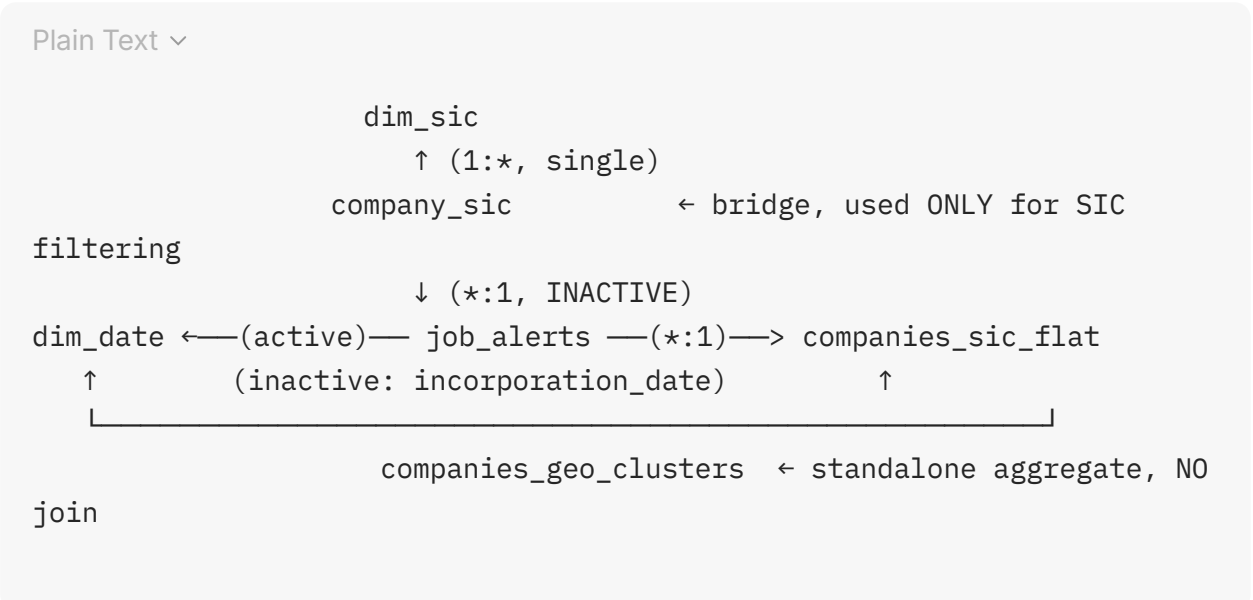
So you now have **four date contexts** fighting each other. Delete all three auto-tables, disable those two relationships, and route `date_received` AND `incorporation_date` through your single `DateTable` (the second one inactive, activated via `USERELATIONSHIP` in measures).

Defect 3: Two Competing Company Dimensions

You import both `companies` AND `companies_sic_flat`, joined M2M on `company_number`. Which one is the source of truth for geography? For name? For incorporation date? Right now: both, ambiguously. Pick **one** as the company dimension. I'd make `companies_sic_flat` the analysis dimension (it has grids + SIC text) and demote `companies` to a reference table or remove it from the report entirely.

The Target Relationship Diagram (No Code — Just the Shape)

Plain Text



Rules this enforces:

- `job_alerts` is the **only fact table** touching dates actively
- `companies_geo_clusters` connects to **nothing** — it's pre-aggregated, filter it directly, don't join it (that M2M grid_key join is causing double-counting)
- `company_sic` is quarantined to SIC filtering only

Your Regression-Proof Checklist (Updated With Your Actual Files)

Before any change, ask which phase you're touching:

- Changing parsing/matching? → Only Postgres + `public job_alerts` refresh needed. Model untouched.
- Changing dedup window? → Only Postgres re-run + refresh. No DAX changes.
- Adding an SIC category? → Only `dim_sic` / `company_sic` refresh. Fact untouched.
- Adding a visual? → Phase 7 only. Nothing below touched.

If a change requires editing **two phases**, your design is coupled — stop and redesign that boundary.

Homework before next step: Tell me — do you want to rebuild the model relationships first (Phase 7 surgery: kill auto-dates, fix cardinality, isolate the cluster table), or lock down the Postgres layer first (confirm `job_alerts` grain = one row per alert, `companies_sic_flat` grain = one row per company)? One of these must be verified before the other makes sense.